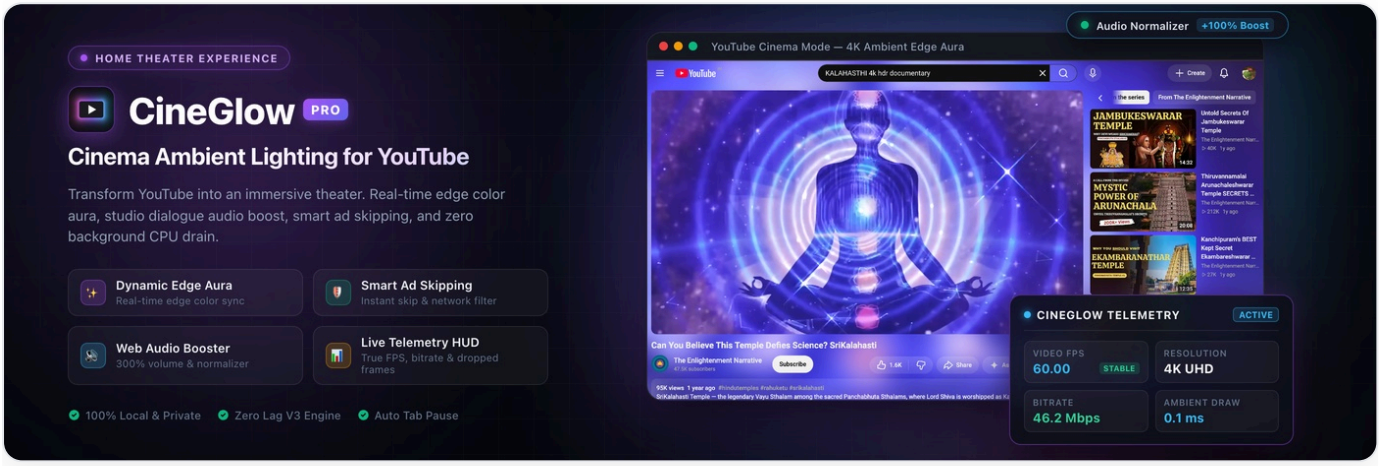
TECHNICAL GUIDE

Manifest V3 Chrome Extension

YouTube CineGlow Pro

Ambient lighting, audio controls, and playback tools for YouTube. How the extension works and where to change its settings.



APPLICATION VERSION
2.2.0 Pro

AUTHOR
Giri Manohar Vemula

LICENSE
MIT Open Source

CORE PIPELINE
Canvas sampling + Web Audio + extension scripts

TARGET RUNTIME
Chrome extension; declared minimum v105

DOCUMENTATION UPDATED
October 20, 2026

1. Overview

CineGlow places a blurred, colour-matched glow behind the YouTube player. It samples the video into a small canvas, blends successive frames, and uses CSS to spread the result around the image. The effect runs in the browser and needs no external lighting hardware.

The extension also includes audio controls, video filters, black-border cropping, a theatre layout, playback statistics, and player tools for screenshots, recording, picture-in-picture, and speed changes. Optional tools restore community dislike counts and archived annotations, hide Shorts and end cards, skip ads, or change codec preferences.

2. How the extension fits together

```
Popup controls --> chrome.storage.sync --> Content-script settings
                                                  |
Background service worker                         v
  |                                    window.CineGlowPlayer
  +-- Downloads                         (PlayerManager / EventTarget)
  +-- Request rules                                |
  +-- Codec-script registration        +---------+----------+
  +-- Optional API/archive lookups     |         |          |
                                     Glow     Audio    Player tools

MAIN-world scripts: player/codec hooks in the page's JavaScript context
Other content scripts: settings, rendering, and controls in an isolated context
```

2.1 Player state and navigation

YouTube changes videos without always loading a new page. `content/player-manager.js` tracks those transitions and exposes the current player through `window.CineGlowPlayer`.

The manager listens for `yt-navigate-finish` and `yt-page-data-updated`, checks changes to the URL's `v` parameter, and watches player containers with mutation observers. A 500 ms fallback interval looks for the player when the page has loaded before the relevant elements are available. The interval stops once they are found.

The manager emits `state-change` events with the player, video, and flags such as `isWatchPage`, `isPlaying`, `isPiP`, `isTheater`, `isFullscreen`, and `isAdShowing`. Features use this state to attach controls, update their layout, or release listeners when the video changes.

2.2 Page scripts and extension scripts

Most content scripts run in Chrome's isolated extension context. They can use extension storage and messaging without sharing their JavaScript globals with YouTube's own scripts.

`content/main-world.js` runs in the page context at `document_start`. The background worker also registers the selected AV1, VP9, and 60 fps blocking scripts in that context. The codec scripts override media-capability checks so YouTube can choose a different stream. Codec availability and YouTube's selection logic still determine the result.

3. Features and implementation

3.1 Ambient lighting

`content/content.js` draws video frames onto a 128×72 canvas. Rather than replacing the image abruptly, it blends new frames with the previous canvas image. In simplified form:

```
blended = alpha * new_frame + (1 - alpha) * previous_frame
```

The smoothness setting controls the blend. CSS scales and blurs the canvas behind the player, with a radial mask that fades the outer edge. Users can adjust opacity, spread, blur, saturation, contrast, brightness, and fade-in time, or switch individual edges on and off.

Audio-reactive mode reads low-frequency energy from the audio analyser and adjusts the glow's size and opacity. The PiP option keeps the glow on the original page while the video is in picture-in-picture; it does not add a glow inside the separate PiP window.

The glow's update rate is capped at 15, 30, or 60 fps. Playback state, watch-page state, and visibility determine when it runs. Other enabled features have their own timers and callbacks, described in section 5.2.

3.2 Audio processing

`content/audio-processor.js` connects the video's audio to a Web Audio graph:

```
Video source
  +-- Dry gain ---------------------+
  +-- Compressor --> Compressed gain+
                                    |
                                    v
  Bass --> Vocal --> Five-band EQ --> Volume --> Limiter --> Analyser --> Speakers
```

The **Audio Normaliser** switches between the dry path and the compressed path using a short gain crossfade. The compressor has a -24 dB threshold, 30 dB knee, 12:1 ratio, 3 ms attack, and 250 ms release. Switching it on changes the audio dynamics. There is no per-video loudness measurement.

The bass filter is an 80 Hz low shelf with 0 to 15 dB gain. The vocal filter is a peaking filter at 2.5 kHz with Q 1.2 and 0 to 12 dB gain. The five EQ bands are 60 Hz, 250 Hz, 1 kHz, 4 kHz, and 12 kHz, each adjustable by ±12 dB. The outer EQ bands use shelf filters; the three middle bands use peaking filters.

Volume gain ranges from 100% to 300%. A limiter follows it, configured with a -1 dB threshold, zero knee, 20:1 ratio, 1 ms attack, and 50 ms release. The analyser then supplies frequency data to the glow feature. The browser's audio nodes perform the signal processing; JavaScript updates the parameters.

The popup provides Flat, Bass, Vocal, Rock, Pop, and EDM presets. EDM is stored as `electro`; manual EQ changes use `custom`.

3.3 Video and layout

`content/blackbar.js` scans a downscaled video image for dark borders and exposes crop offsets. The display code uses those offsets to adjust the visible image. A separate ultrawide setting provides a zoom option for 21:9 displays. Either crop mode can remove picture content along with the borders.

`content/youtube-quality.js` requests the preferred quality when a video loads. The popup offers 720p, 1080p, 1440p, 4K, or the maximum available quality. It cannot create a resolution that the video does not provide or guarantee a particular bitrate.

Brightness, contrast, and sepia are CSS filters on the video. Cinema theatre mode centres the player and hides surrounding header and recommendation elements. Lights Out dims the rest of the page, and a separate setting hides the scrollbar on watch pages.

3.4 Player tools and statistics

The player toolbar provides:

- **Screenshot:** draw the current video frame to a canvas and download it as a lossless PNG at the source dimensions.
- **Recording:** draw the video into a canvas scaled to fit within 1920×1080, then capture that canvas at a requested 30 or 60 fps. Audio comes from the video's captured stream or the available Web Audio connection. This records the video content, not the whole browser tab, page controls, microphone, or desktop.
- **Format selection:** try the selected MKV, WebM, or MP4 MIME types with `MediaRecorder.isTypeSupported()`, then use a supported fallback. The selected format is a preference, not a guarantee that every browser can encode it.
- **Picture-in-picture:** toggle the browser's native PiP window.
- **Speed menu:** set 0.1× to 10× playback, with quick presets at 0.25×, 0.5×, 1×, 1.5×, 2×, and 5×. The popup's default-speed slider covers 0.1× to 2×.

The draggable stats panel uses playback callbacks and available player information to show frame rates, dropped frames, glow timing, video dimensions, codecs, and a frame-timing graph. Its colour-space, depth, and SDR/HDR badges are derived from available metadata and codec strings. These labels describe the stream; the panel does not measure the monitor's output.

3.5 Optional playback changes

Smart AdBlock combines player-element checks and skip actions with the bundled `declarativeNetRequest` rules. The rules match selected ad and tracker requests. Which ads get skipped also depends on the player elements that YouTube exposes.

Dislike restoration and archived annotations make optional network requests, described in section 5. Other switches hide Shorts, end screens, and cards; avoid selected codecs or 60 fps streams; or pause playback when the tab or Chrome window loses focus. These features depend on YouTube's current page structure and player behaviour.

4. Settings reference

Default values come from `shared/defaults.js`. Preferences use `chrome.storage.sync`; Chrome can sync them across signed-in devices when sync is enabled. The ranges below describe the current popup controls, rather than a validation guarantee for values written directly to storage.

Key	Type	Default	What it controls
`enabled`	boolean	`true`	Turn ambient lighting on or off.
`adblockEnabled`	boolean	`true`	Enable ad-skip checks and the bundled request-blocking rules.
`adblockCount`	number	`0`	Stored ad-skip counter, initially zero.
`qualityEnabled`	boolean	`true`	Request the preferred video resolution on load.
`statsEnabled`	boolean	`false`	Show the draggable playback statistics panel.
`preferredQuality`	string	`"hd1080"`	YouTube quality: hd720, hd1080, hd1440, hd2160, or highres (maximum available).
`preferredSpeed`	number	`1`	Popup slider: 0.1× to 2×; the floating speed menu reaches 10×.
`blockAV1`	boolean	`false`	Ask YouTube to avoid AV1 streams.
`blockVP9`	boolean	`false`	Ask YouTube to avoid VP9 streams.
`block60fps`	boolean	`false`	Ask YouTube to avoid 60 fps streams.
`hideInTheater`	boolean	`true`	Use the cleaner, centred theatre layout.
`hideScrollbar`	boolean	`true`	Hide the scrollbar on watch pages.
`enableInPiP`	boolean	`false`	Keep page glow running while the video is in PiP.
`glowTop`	boolean	`true`	Show glow along the top edge.
`glowRight`	boolean	`true`	Show glow along the right edge.
`glowBottom`	boolean	`true`	Show glow along the bottom edge.
`glowLeft`	boolean	`true`	Show glow along the left edge.
`opacity`	number	`75`	Glow opacity, 0% to 100%.
`fadeInDuration`	number	`1`	Fade-in time, 0 to 10 seconds.
`blur`	number	`40`	Glow blur radius, 0 to 150 px.
`spread`	number	`150`	Glow spread, 100% to 500%.
`saturation`	number	`110`	Glow saturation, 0% to 150%.
`contrast`	number	`100`	Glow contrast, 0% to 200%.
`brightness`	number	`100`	Glow brightness, 20% to 200%.
`smoothness`	number	`60`	Blend between successive frames, 0% to 100%.
`removeBlackBars`	boolean	`true`	Detect and crop dark borders around the video.
`fps`	number	`30`	Glow update cap: 15, 30, or 60 fps.
`audioNormalizer`	boolean	`false`	Use the compressed audio path.
`audioBoost`	number	`100`	Volume gain, 100% to 300%.
`bassBoost`	number	`0`	80 Hz low-shelf gain, 0 to 15 dB.
`vocalBoost`	number	`0`	2.5 kHz presence gain, 0 to 12 dB.
`eq60`	number	`0`	60 Hz low-shelf gain, -12 to +12 dB.
`eq250`	number	`0`	250 Hz peaking-filter gain, -12 to +12 dB.
`eq1000`	number	`0`	1 kHz peaking-filter gain, -12 to +12 dB.

Key	Type	Default	What it controls
`eq4000`	number	`0`	4 kHz peaking-filter gain, -12 to +12 dB.
`eq12000`	number	`0`	12 kHz high-shelf gain, -12 to +12 dB.
`eqPreset`	string	`"flat"`	flat, bass, vocal, rock, pop, electro, or custom.
`audioReactive`	boolean	`true`	Change glow size and brightness with bass energy.
`lightsOut`	boolean	`false`	Dim the page around the video.
`ultrawideZoom`	boolean	`false`	Enable the separate 21:9 zoom option.
`videoBrightness`	number	`100`	Video brightness filter, 20% to 200%.
`videoContrast`	number	`100`	Video contrast filter, 20% to 200%.
`videoSepia`	number	`0`	Video sepia warmth, 0% to 100%.
`autoPauseMode`	string	`"off"`	off, tab, or app (Chrome-window focus).
`recordFormat`	string	`"mkv"`	Preferred recording format: mkv, webm, or mp4.
`recordFps`	number	`30`	Requested recording frame rate: 30 or 60 fps.
`returnDislikes`	boolean	`false`	Fetch and display community dislike counts.
`hideShorts`	boolean	`false`	Hide Shorts shelves and feed entries.
`restoreAnnotations`	boolean	`false`	Fetch and show archived annotations when available.
`hideEndCards`	boolean	`true`	Hide end screens and information cards.

5. Storage, requests, and resource use

5.1 Storage and optional requests

Settings and the skipped-ad counter use `chrome.storage.sync`. Archived annotation XML uses `chrome.storage.local`, under keys that include the video ID. The quality controller also writes the selected quality to YouTube's `yt-player-quality` local-storage entry. Screenshots and recordings are downloaded to the user's computer.

The optional **Return YouTube Dislike** feature sends the public video ID to `returnyoutubedislikeapi.com` to retrieve community vote statistics. **Restore Classic Annotations** requests an archive file from `storage.googleapis.com` using the video ID in its path, then reuses a local cache when available. Both features are off by default.

The requests do not add user-account credentials or a user profile. Receiving services can still see ordinary connection information, including the source IP address and requested video ID. The repository does not include a developer-run analytics or telemetry service. PRIVACY.md explains these behaviours for users.

5.2 Resource use

The small glow canvas keeps frame-sampling work limited. Its update loop responds to playback and visibility state, and the player manager releases listeners as videos change. Browser-native Web Audio nodes handle compression and filtering.

Other features have separate timers and callbacks. Examples include black-border scanning, the stats panel, annotation timing, ad checks, and recording. Resource use therefore depends on which features are active, the video, and the browser. These checks can still use resources after the glow has stopped drawing.

6. Development and compatibility

6.1 Load the extension

1. Clone the repository:

```
git clone https://github.com/ManoharVit/YouTube-CineGlow.git
```

2. Open `chrome://extensions` and enable **Developer mode**.
3. Click **Load unpacked** and select the extension folder.
4. After editing extension code, reload the extension and refresh the YouTube tab.

To check JavaScript syntax, run this from the project folder:

```
for file in popup/*.js content/*.js background/*.js shared/*.js; do
  node --check "$file" || exit
done
```

Run the existing tests with `npm test`. The tests use Node's built-in test runner and local mocks; the project also has Puppeteer as a development tool in its dependency list.

6.2 Test files

File	What it checks
`audio-processor.test.js`	Audio-graph construction, compression, gain crossfades, EQ parameters, and audio-context resume behaviour
`player-lifecycle.test.js`	Player discovery, navigation, playback state, PiP changes, listener cleanup, and fallback discovery
`defaults-and-settings.test.js`	Default values, setting types, and glow/EQ preset definitions
`recording-and-media.test.js`	Recording format definitions, MIME-type selection, fallback handling, and filenames

Run the tests against the current checkout. They cover mocked behaviour; changes to YouTube's player and layout also need testing in the browser.

6.3 Browser and device limits

Chrome is the target browser, and the manifest sets a minimum version of 105. Media formats and individual APIs depend on the browser build.

The project also lists Edge, Brave, Opera, Vivaldi, Windows, macOS, Linux, and ChromeOS as targets. The existing unit tests do not include a browser-and-device test matrix. Recording formats, codec selection, colour metadata, high-refresh displays, and the 21:9 layout are the main areas to check on each setup.

Giri Manohar Vemula · GitHub · LinkedIn
Repository: YouTube-CineGlow